

Using Git for Research

Lucy M Thomas
lmthomas@caltech.edu

LIGO SURF
24th June 2024



Who Am I?



- Postdoctoral researcher in gravitational waveform modelling and LIGO data analysis
- I am from London UK, did my PhD in Birmingham UK before coming to Caltech last September
- ✧ • I am not a Git super-expert! Just a researcher who has learned the hard way that Git can be extremely useful...
- My office is West Bridge 264 (next to the SURF room!), feel free to come in and say hi, we have candy :)



Lucy (she/her)

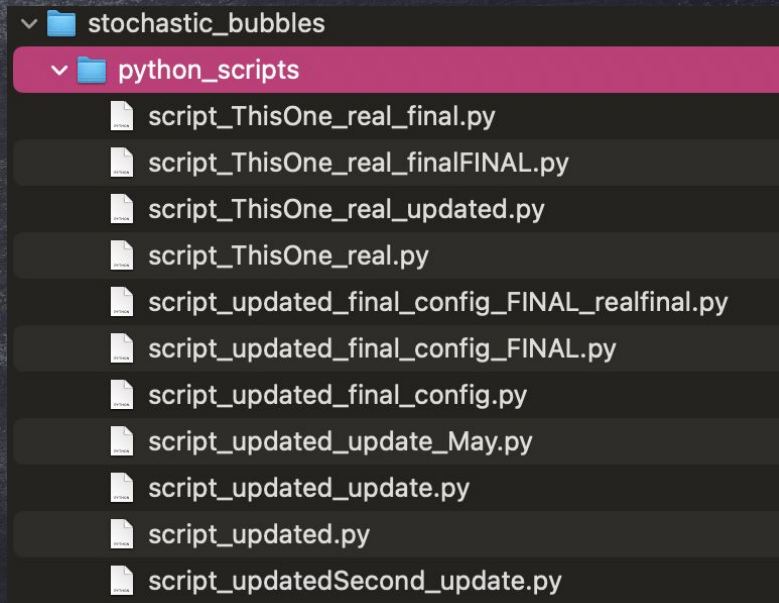


Once Upon A Time...



Me as an undergrad,
wearing a trashbag

Too many versions of
the same code for
my final undergrad
research project



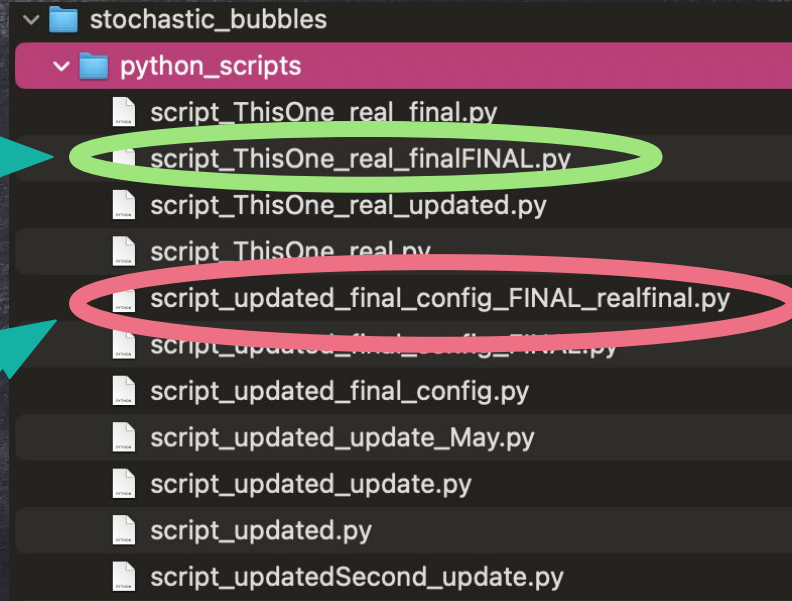
Once Upon A Time...



The code I should
have submitted



The code I actually
submitted, complete
with this line and an
empty section:



This bit is tricky, DO LATER ON!!!!

Once Upon A Time...

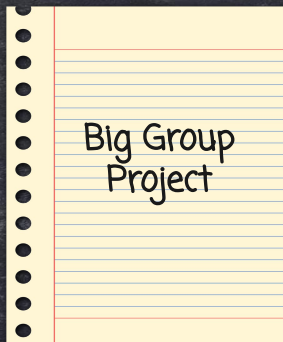
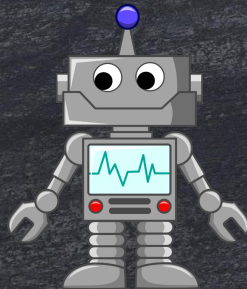
This story is not
unique to me...



Another Scenario...

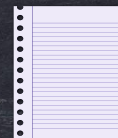
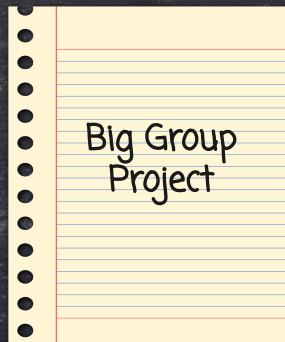
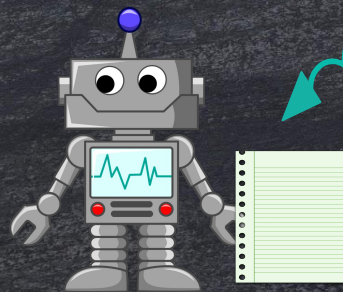


Let's say these four people are working on a group project where they are editing the same file



Another Scenario...

Each will end up with their own version of the file, with the changes they have made individually



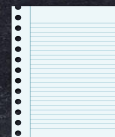
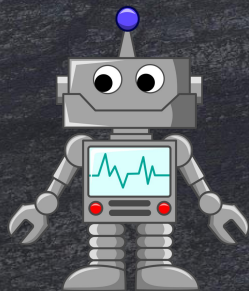
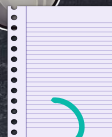
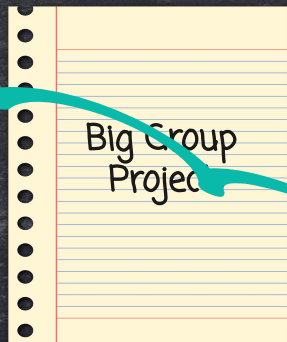
Another Scenario...

Either they have to regularly update each other, sending over the changes they've made...



Another Scenario...

...or they have to work on different sections, then send the final versions to one person to merge into a single file.



How can we make this system better?



The Solution:

verSion control with Git!

What Is Git?

- Created by Linus Torvalds (who also created Linux) in 2005
- Designed to do version control on Linux kernel, but is now used extremely widely on all operating systems
- It is a distributed version control system (VCS), meaning all copies of the repository, local and remote, are the same and every one is "just as good" as every other
- (A "git" is a grumpy old man, particularly in English slang)



Why IS Git Good?

- It's space-efficient
- It's safe
- It allows you to see a legible timeline of your work
- It allows you to backup your work remotely
- You can experiment easily using branches without breaking your code
- It makes collaboration much easier
- It's not just for code! Other files can be tracked too



How does Git work?



Let's say you have project on your laptop (local machine), or on a cluster, called `my_project`

How Does Git Work?

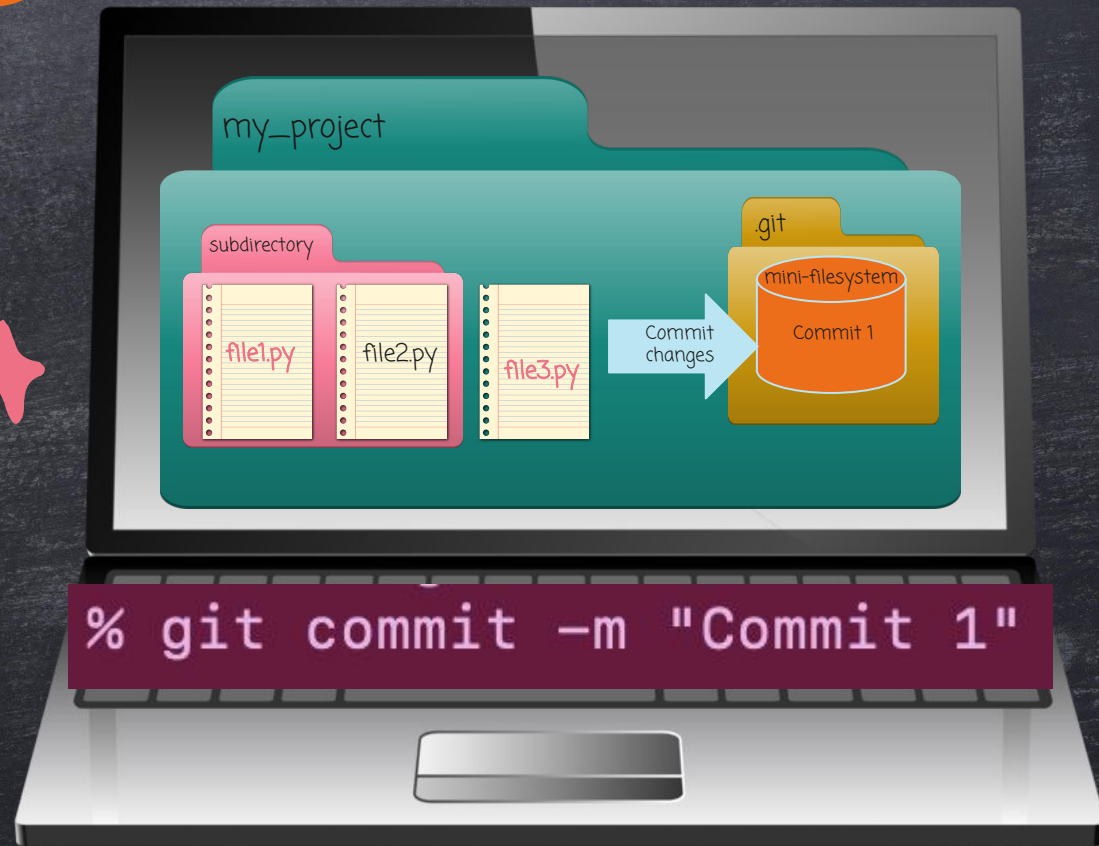


If we make this project into a Git-tracked project (a Git repository), an extra directory gets added called `.git`, which points to a mini-file-system

The mini-file-system contains your data, and things needed to make Git work for this project (it is complicated, and not designed to be human-readable)

← Pink text shows bash commands typed in the terminal/shell

How Does Git Work?



You can then make changes to your local files, including adding and deleting files, but this won't change the mini-file-system, which is now behind

We need to 'commit' our changes, so that they can be pushed into the mini-file-system



How Does Git Work?



```
% git commit -m "Commit 1"
```

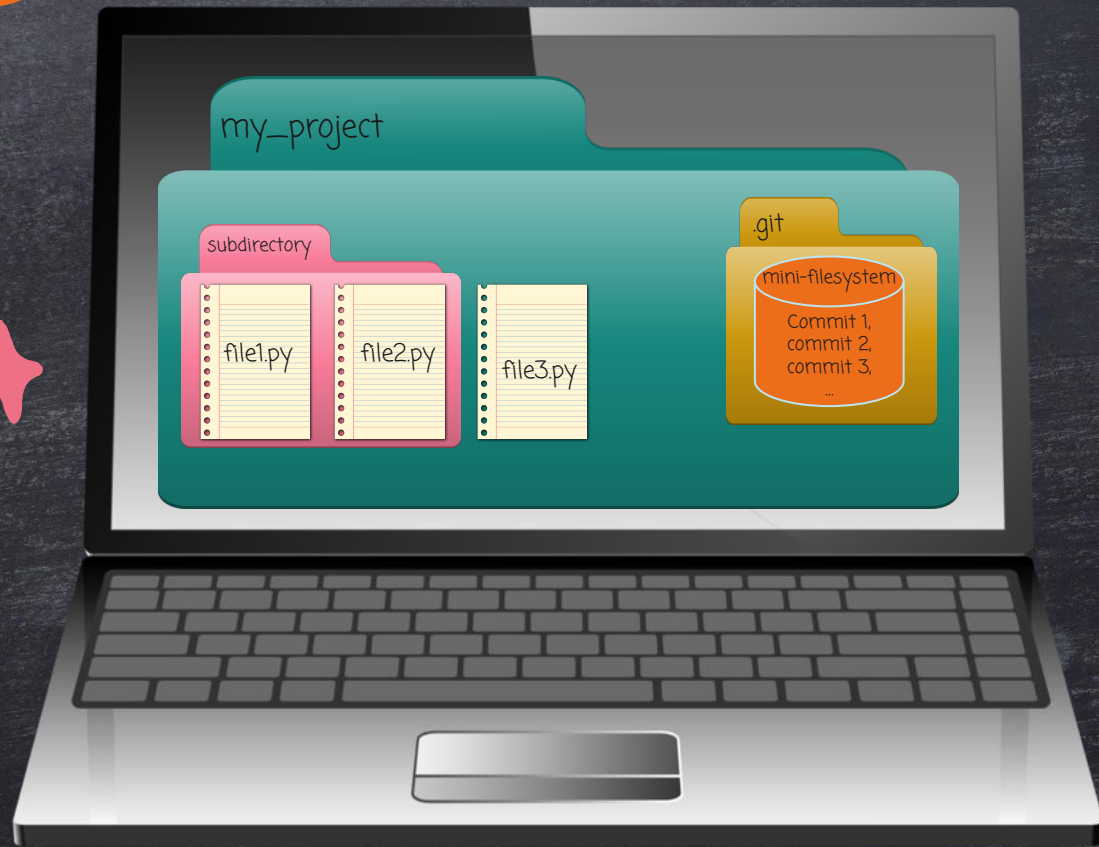
The history of a project or file is represented by a series of commits.

Each commit contains:

- The current versions of tracked files
- Pointers to commits immediately before it (**parent commits**)
- A **commit message**, which you write to describe what changes were made (the bit in quotes after the -m flag above)
- An **identifying hash** (collection of numbers and letters)
- Author name and email
- Date and time of commit



How Does Git Work?

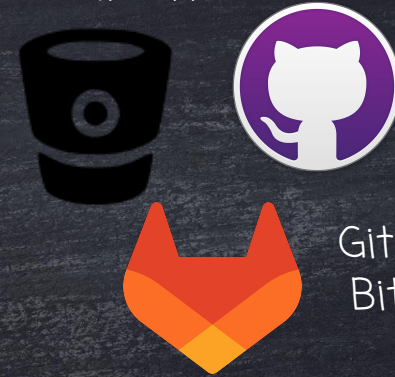
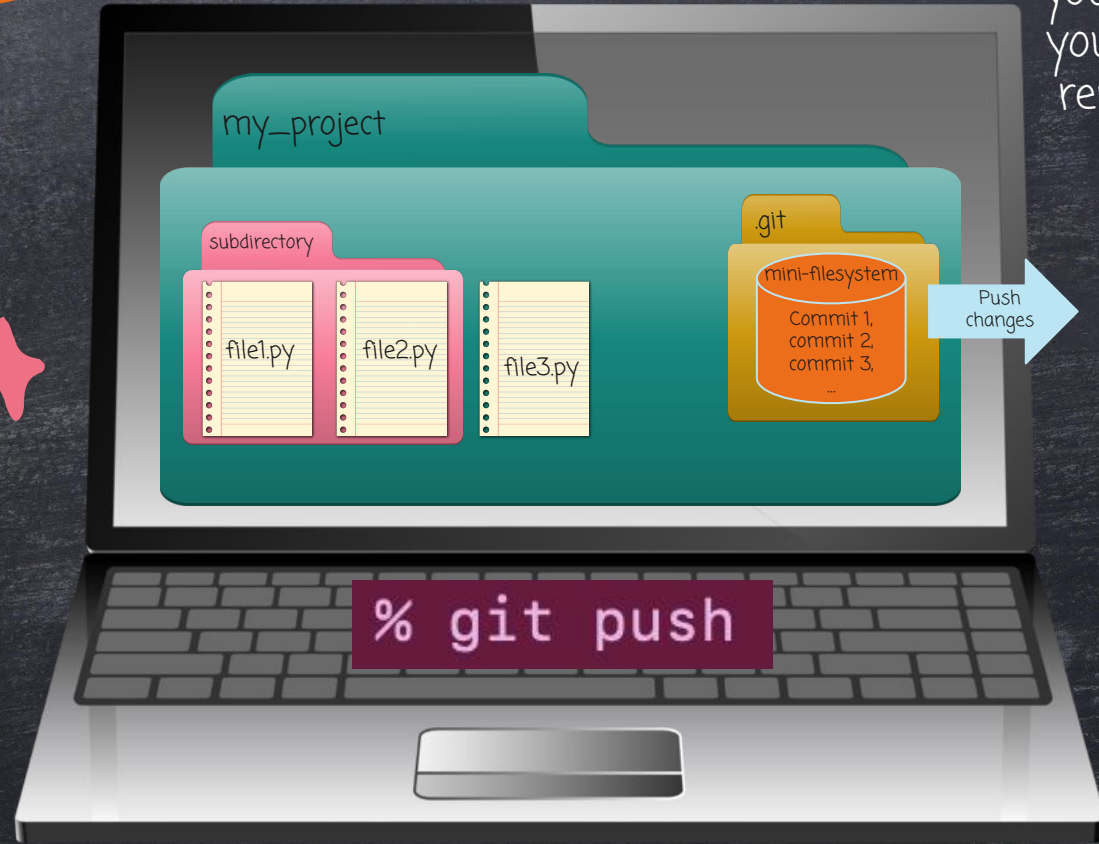


The mini-file-system is append-only, so you can always go back and look at the full history of changes

But all of this is still on your local machine or on the cluster, and we want our data to be backed up in a different location, where it is accessible to collaborators

How Does Git Work?

At the end of a work session, you should 'push' and save your changes to a remote repository, hosted on the internet

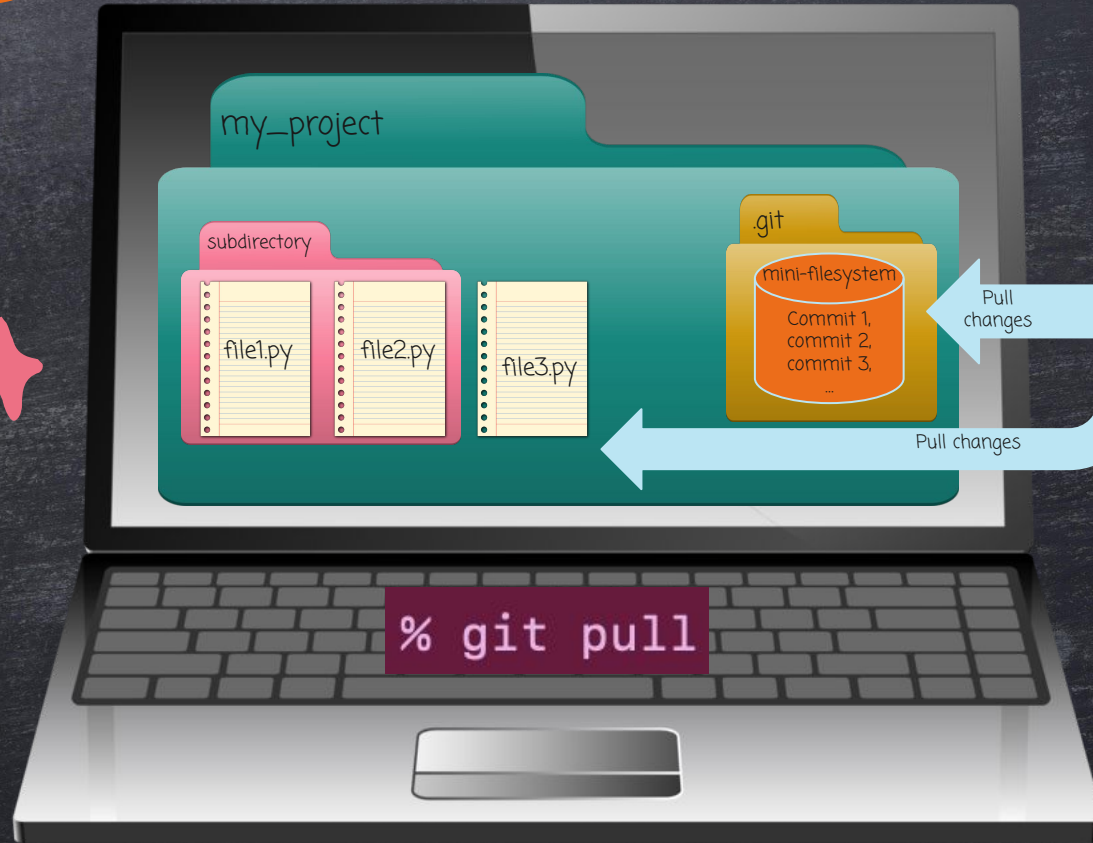


GitLab, GitHub, Bitbucket etc.

Now you and others (if you wish) can see the history of the project and retrieve your work from this remote repository

How Does Git Work?

To retrieve your data from this remote repository, use `git pull` to put the data in your mini-filesystem and local files



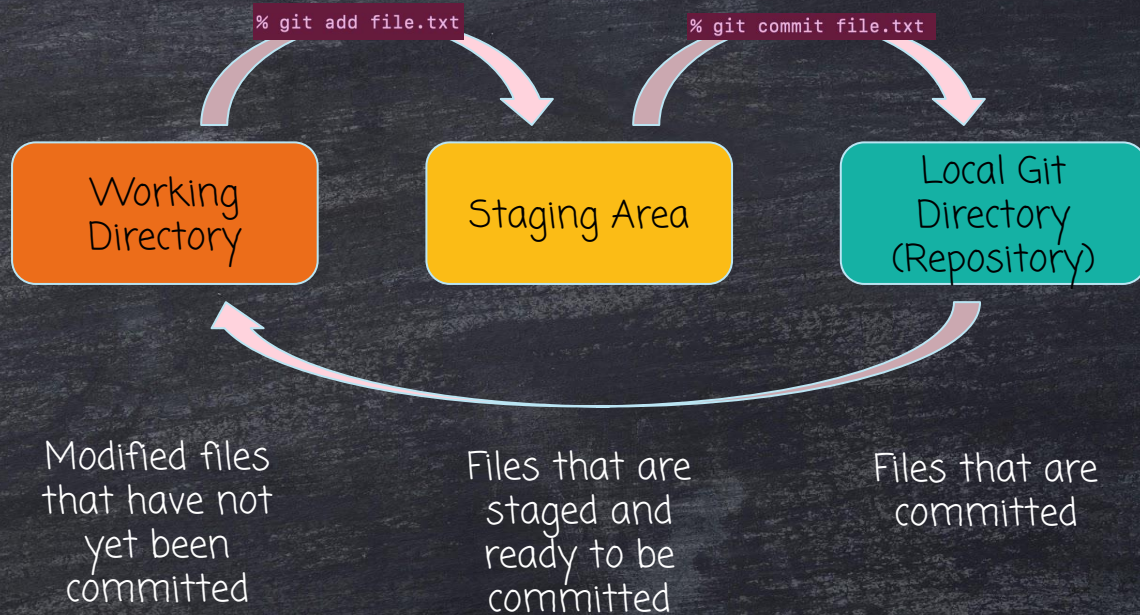
If your local files have changed, git will merge the changes if possible. If not, there will be an error and you will have to resolve conflicts



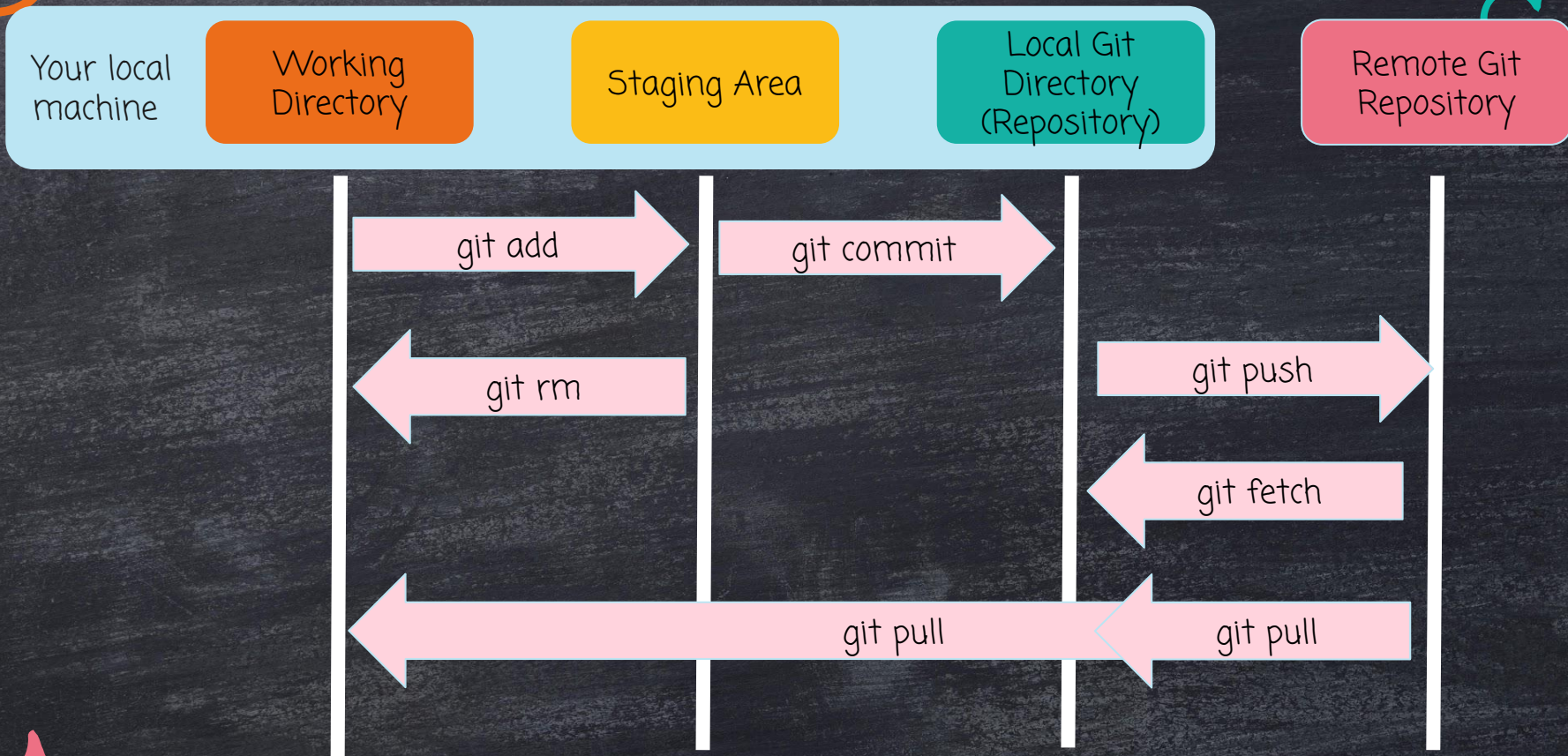
How Does Git Work?



Git uses a "three tree" model to track files. All files on your local machine being tracked by Git will be in one of three states:



How Does Git Work?



Some Useful Basic Git Commands

<code>git init</code>	Create a new Git repository within the current directory that already contains files
<code>git add FILENAME</code>	Add changes to the file called FILENAME to the staging area
<code>git commit -m "MESSAGE"</code>	Create a new commit from the currently staged changes
<code>git commit -m "MESSAGE" --amend</code>	Instead of making new commit, add staged changes to the last commit
<code>git status</code>	Give a summary of files that have been modified and/or staged
<code>git diff</code>	View unstaged file changes line by line
<code>git log</code>	View a list of commits
<code>git stash</code>	Save the current state of the working tree to a stack and reset the working tree to the latest commit



Branches

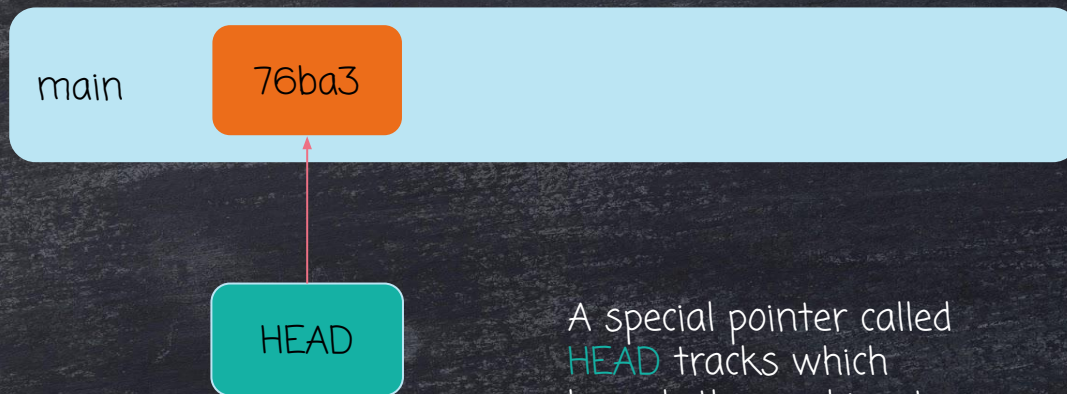


If you wish to experiment with making a change (for example if you want to add a feature to your code but are not sure if it will break the pre-existing functionality), you can start a new branch



This lets you diverge in development from the main project history. We can think of branches as timelines

When you first make a **commit**, you start on the **main** branch



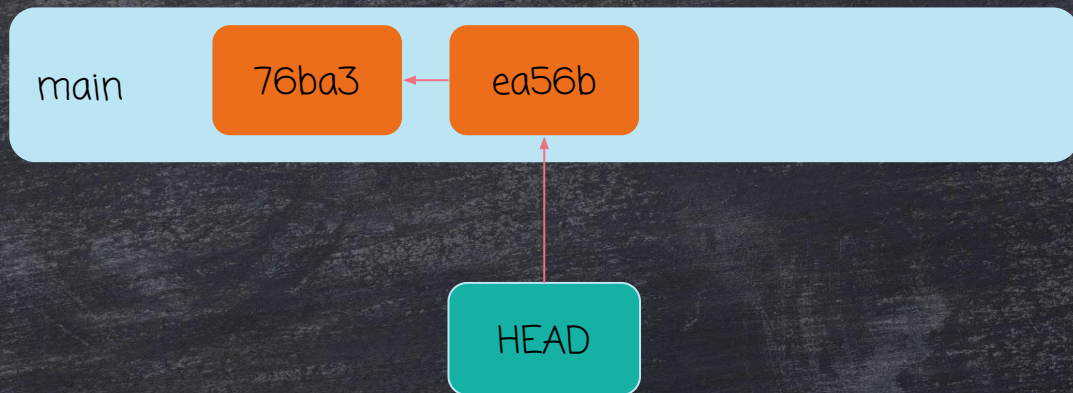
A special pointer called **HEAD** tracks which branch the working tree is on



Branches



Committing moves the
current branch and HEAD
forward by one commit



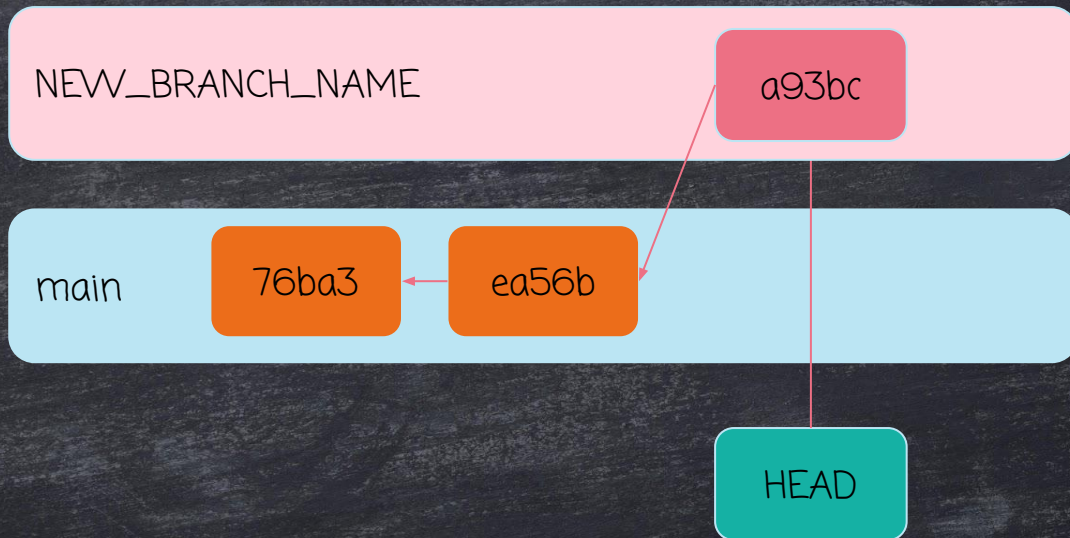


Creating Branches

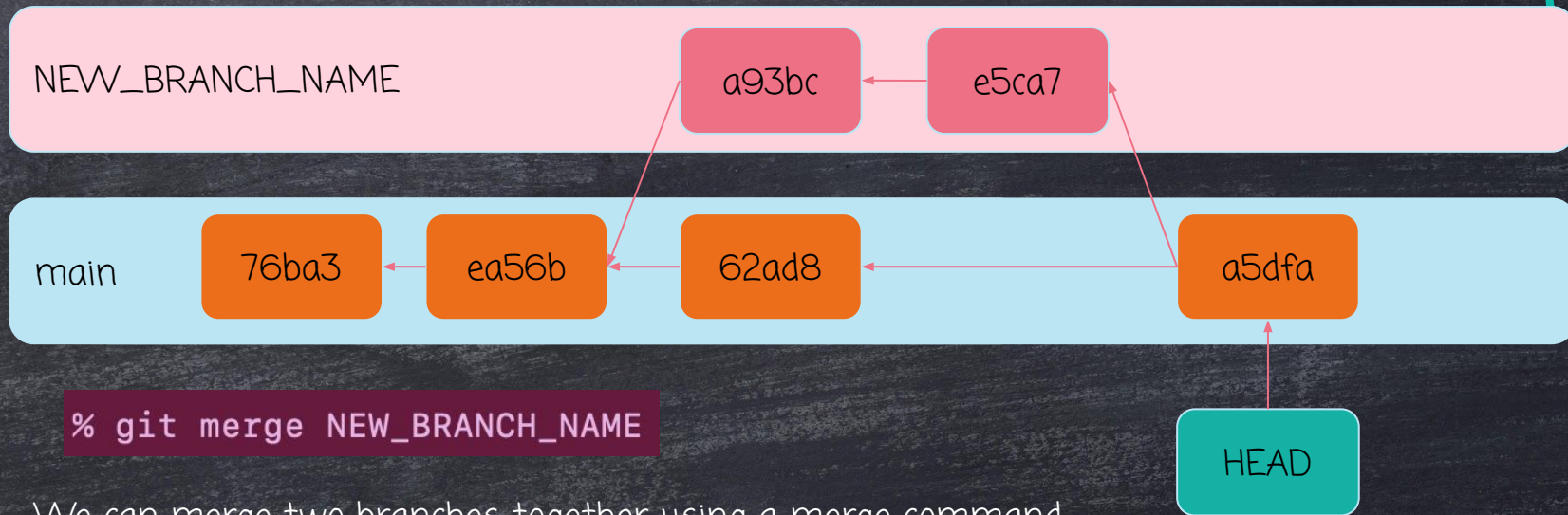


```
% git checkout -b NEW_BRANCH_NAME
```

Create a new branch and switch HEAD to it, now any commits we make will be inside the new branch



Combining Git Branches: Merge

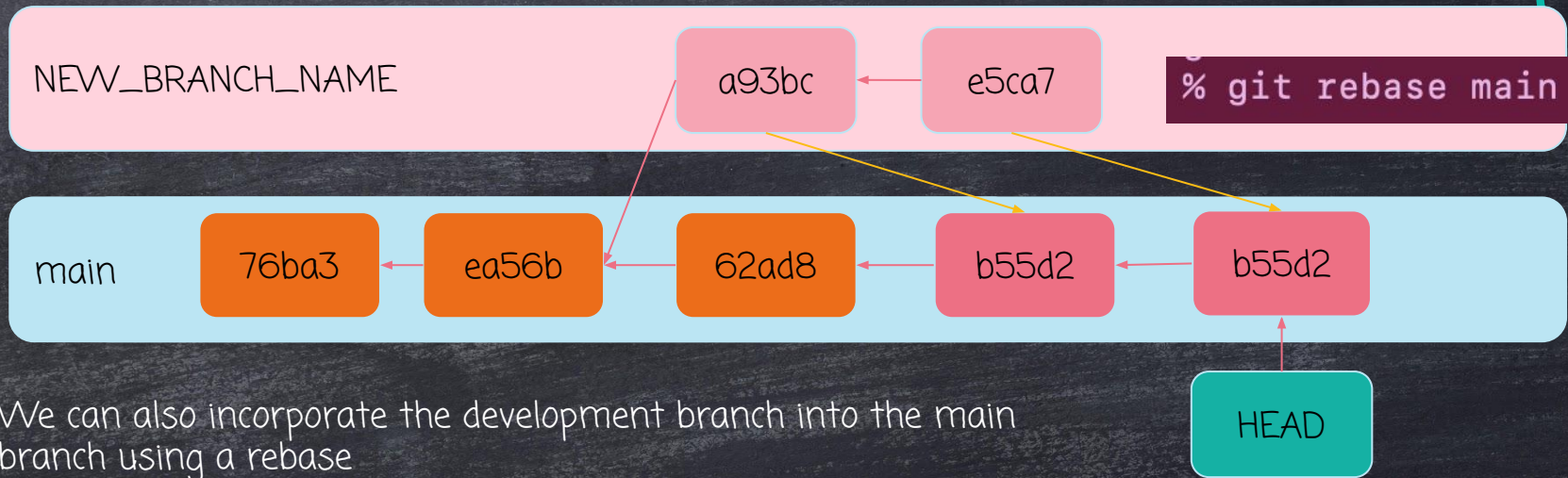


We can merge two branches together using a merge command

If the branch histories don't diverge, Git will do a fast-forward merge. This will move the current branch and HEAD forward to the feature branch

If they branch histories diverge, the current branch will have a commit added obtaining the development branch. There may be a merge conflict, which will need to be solved

Combining Git Branches: Rebase



We can also incorporate the development branch into the main branch using a rebase

First, HEAD moves to the tip of the main branch, then Git tries to apply each new commit from the development bench in sequence. The rebased commits have new hashes, so they're different even though they contain the same changes.

If there is a conflict, Git will pause the rebase so you can fix it.

Both merge and rebase give a united commit history, but rebasing gives a linear commit history, where every commit has only one parent and can be dangerous because it "rewrites history".



Some Useful Branch Commands



<code>git branch</code>	Show the current branch along with a list of all branches
<code>git checkout -b BRANCH_NAME</code> OR <code>git switch -c BRANCH_NAME</code>	Move HEAD to a new branch. Uncommitted changes are preserved
<code>git checkout BRANCH_NAME</code> OR <code>git switch BRANCH_NAME</code>	Move HEAD to another branch and change working tree files accordingly. Won't work if you have uncommitted changes in a file that would be affected
<code>git merge DEVELOPMENT_BRANCH</code>	Create a new commit on the current branch that includes changes from the tip of the development branch
<code>git rebase BASE_BRANCH</code>	Reapply (rebase) the commits of the current branch on top of the base branch



Branches Etiquette



- Commits and branches should be as modular as possible: group related changes together
- Avoid committing directly to the main branch outside of merging/rebasing: make big code changes on a development or testing branch first
- Commit messages should follow this template:



Short summary of changes in less than 50 characters.

A blank line, followed by a detailed description of changes if necessary, Line wrap after 72-80 characters.

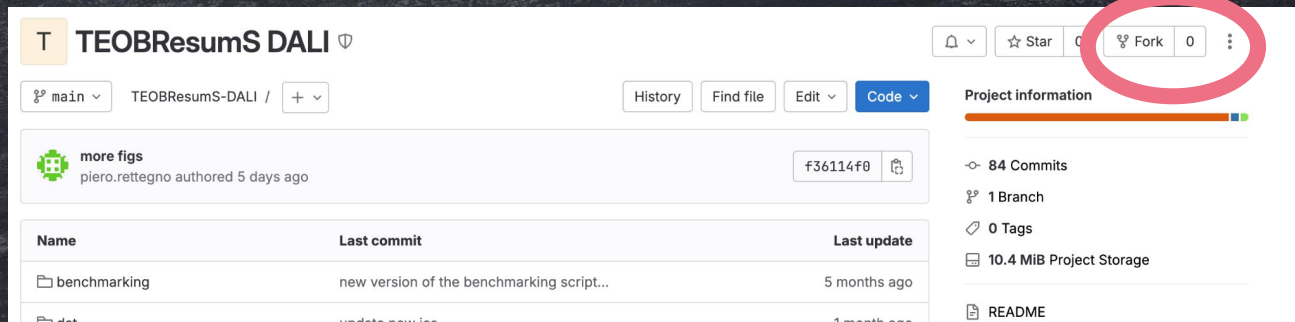
Forking Projects

A fork is a new repository that shares code and visibility settings with the original "upstream" repository

Use a fork when you want to use someone else's code repository as a starting point for your own work, without necessarily pushing back to the original repository

You can keep the forked repository in sync with the original project by:

- Adding the original project as a remote repository
- Fetching regularly from that original project
- Rebase your current development on top of the branch of interest that you have updated from the fetch



The screenshot shows the GitHub interface for the repository 'TEOBResumS DALI'. The 'Fork' button is highlighted with a red circle. The repository has 84 commits, 1 branch, and 0 tags. The project information section shows 10.4 MiB of project storage and a README file. The commit history table is partially visible.

Name	Last commit	Last update
benchmarking	new version of the benchmarking script...	5 months ago
dat	update new.js	1 month ago

Getting Started: You will Need



1. Make sure **Git** is installed on your local machine (or wherever your local repository will live)
2. Make sure you are able to **log into the remote client** (we recommend you use the LIGO GitLab instance, but GitHub, other GitLab instances, and BitBucket are also fine)
3. Make sure your **SSH keys are configured** so that the machine you want to develop code on (your local machine or the cluster) is able to push changes to the remote
4. **Create your Git-tracked project** and go!
5. (BONUS: If you use **VSCode**, make sure you have configured it to work with Git)



1. Installing Git To Your Machine ✨

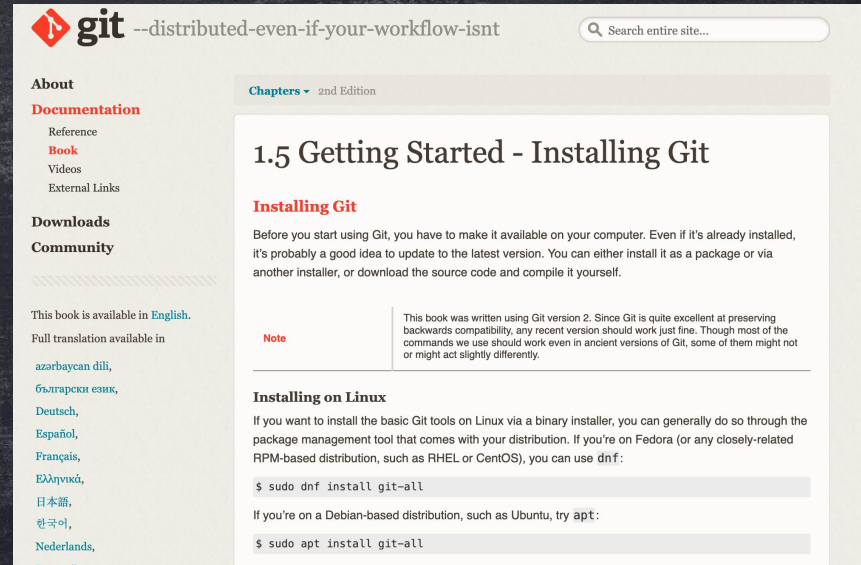
First open up your terminal or shell and check if you already have Git installed:

```
% git --version
```

It should print out something like (the specific version of git doesn't matter too much)

```
git version 2.39.3 (Apple Git-146)
```

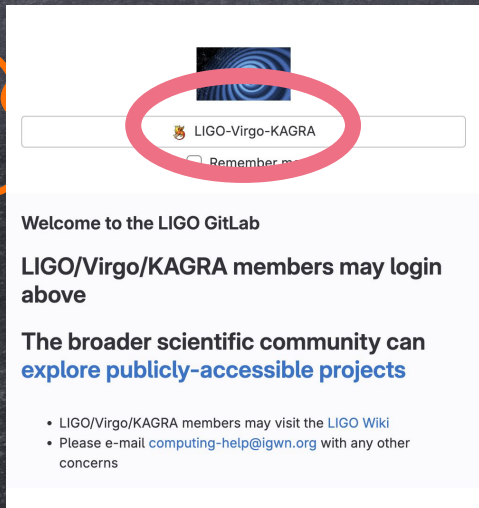
If not, here are some Git installation instructions:
<https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>



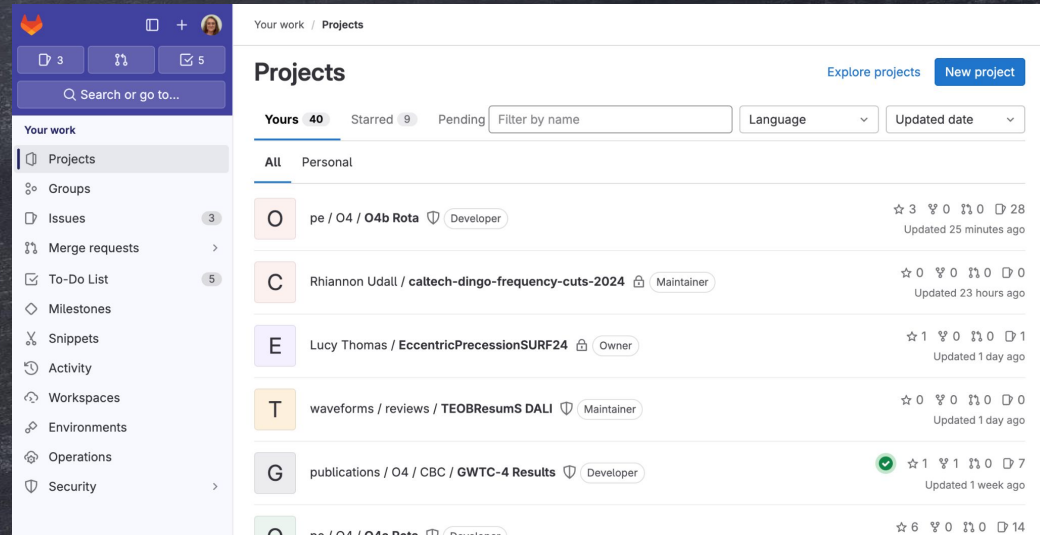
The screenshot shows the Git website with the following content:

- git** --distributed-even-if-your-workflow-isnt
- About**
- Documentation**
 - Reference
 - Book**
 - Videos
 - External Links
- Downloads**
- Community**
- Chapters** ▾ 2nd Edition
- 1.5 Getting Started - Installing Git**
- Installing Git**
- Before you start using Git, you have to make it available on your computer. Even if it's already installed, it's probably a good idea to update to the latest version. You can either install it as a package or via another installer, or download the source code and compile it yourself.
- Note**
- This book was written using Git version 2. Since Git is quite excellent at preserving backwards compatibility, any recent version should work just fine. Though most of the commands we use should work even in ancient versions of Git, some of them might not or might act slightly differently.
- Installing on Linux**
- If you want to install the basic Git tools on Linux via a binary installer, you can generally do so through the package management tool that comes with your distribution. If you're on Fedora (or any closely-related RPM-based distribution, such as RHEL or CentOS), you can use `dnf`:
- ```
$ sudo dnf install git-all
```
- If you're on a Debian-based distribution, such as Ubuntu, try `apt`:
- ```
$ sudo apt install git-all
```

2. Logging In To A Remote Client ✨



Login with
your ligo.org
credentials



<https://git.ligo.org/>

This is the LIGO-hosted remote repository client, which is an instance of GitLab. If you want to collaborate on a project with other LIGO members, we recommend you use it! GitHub is another alternative ✨

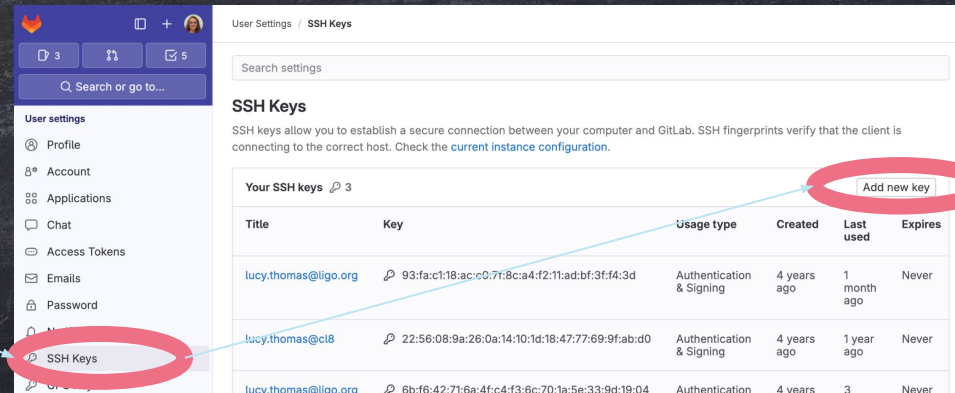
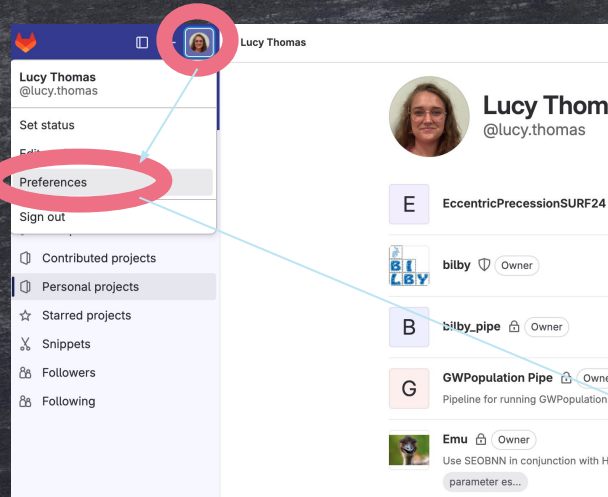


3. Configuring SSH keys

You may need to add SSH keys to communicate securely between your local machine and the remote repository, instructions here:

<https://docs.gitlab.com/ee/user/ssh.html#add-an-ssh-key-to-your-gitlab-account>

If you haven't yet generated an SSH key pair on your local machine, got back to Rhiannon's tutorial on Friday before doing this!



3. Configuring SSH keys

On your local machine, copy the contents of your public key to clipboard using one of these three commands in your terminal/shell

macOS

```
tr -d '\n' < ~/.ssh/id_ed25519.pub | pbcopy
```

Linux (requires the xclip package)

```
xclip -sel clip < ~/.ssh/id_ed25519.pub
```

Git Bash on Windows

```
cat ~/.ssh/id_ed25519.pub | clip
```

Change the filename to whatever your public key is called

Then back in the web browser...

SSH Keys

SSH keys allow you to establish a secure connection between your computer and GitHub. SSH fingerprints verify that the client is connecting to the correct host. Check the [current instance configuration](#).

Your SSH keys 4

Add an SSH key

Add an SSH key for secure access to GitHub. [Learn more](#).

Key

Paste the clipboard contents into here

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'.

Title

Example: MacBook key

Key titles are publicly visible.

Usage type

Authentication & Signing

Expiration date

2025-06-24

Optional but recommended. If set, key becomes invalid on the specified date.

Add key

Cancel

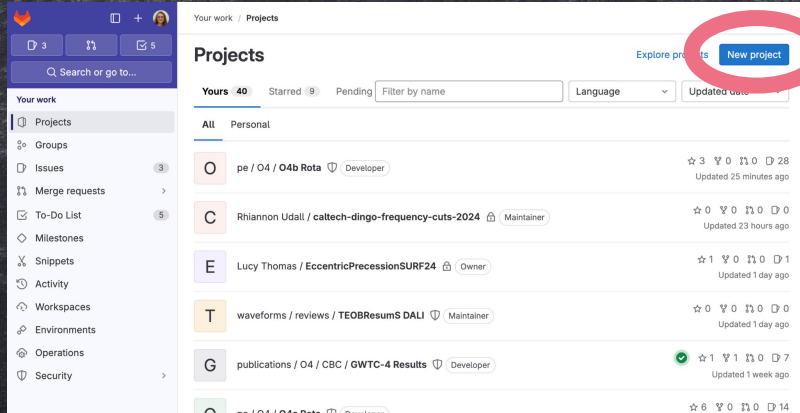
Give the key a title, which should probably include information about your machine eg. "Lucy's Macbook Air M2 2022"

4. Creating A Project



There are multiple ways to create a Git-tracked project, for example:

- Create a local repository first, then link it to a remote repository that you create (git init)
- Create the repository remotely first, then clone it to your local machine to begin working on it (except in a few special circumstances, we can't make changes directly to a remote repository) (new project button, git clone URL)
- Clone a pre-existing repository that someone else owns but you have access to
- Fork a pre-existing repository that you do not have push access to, to create your own version



```
% git init
```

```
% git clone URL
```



4. Creating A Project: Example



- We will create a project locally, then push it to the remote LIGO GitLab
- To follow along, execute the following commands in a new terminal/shell:

```
(base) lucythomas@MacBook-Air-184 ~ % pwd  
/Users/lucythomas
```

First let's check which directory we are currently in

```
(base) lucythomas@MacBook-Air-184 ~ % mkdir test_git_repo  
(base) lucythomas@MacBook-Air-184 ~ % cd test_git_repo
```

We make a new directory called `test_git_repo`, and then move into it

```
(base) lucythomas@MacBook-Air-184 test_git_repo % git init  
Initialized empty Git repository in /Users/lucythomas/test_git_repo/.git/
```

We initialise Git in this repository

```
(base) lucythomas@MacBook-Air-184 test_git_repo % git remote add origin git@git.ligo.org:lucy.thomas/test_git_repo.git
```

We set where the upstream remote repository is

Change this to your name

Set this to whatever you want your remote repository project to be called, keeping the .git



4. Creating A Project: Example



```
(base) lucythomas@MacBook-Air-184 test_git_repo % touch test_file.txt
```

We create an empty file inside our working directory, called test_file.txt (if using Windows, the touch command won't work, instead use "type nul > test_file.txt")

```
(base) lucythomas@MacBook-Air-184 test_git_repo % git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
  (use "git add <file>..." to include in what will be committed)
```

```
    test_file.txt
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

If we run git status, we see that we are on the "main" branch, which was created automatically when we ran "git init". We have not yet committed anything, and we have one file which is not yet tracked.



4. Creating A Project: Example



To track our new file, we add it to the staging area using "git add"

```
(base) lucythomas@MacBook-Air-184 test_git_repo % git add test_file.txt
(base) lucythomas@MacBook-Air-184 test_git_repo % git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   test_file.txt
```

Running git status again, we can see that our changes to the new file are now staged for commit



4. Creating A Project: Example



```
(base) lucythomas@MacBook-Air-184 test_git_repo % git commit -m "First commit, added test_file.txt"
[main (root-commit) a3a6558] First commit, added test_file.txt
Committer: Lucy Thomas <lucythomas@MacBook-Air-184.local>
```

We make our first commit! Remember to include a commit comment describing the change

```
(base) lucythomas@MacBook-Air-184 test_git_repo % git config user.name "Lucy Thomas"
(base) lucythomas@MacBook-Air-184 test_git_repo % git config user.email lucy.thomas@ligo.org
```

Before we push these changes to the remote repository, we want to make sure we have configured our local repository to include the correct name and email address



4. Creating A Project: Example



```
(base) lucythomas@MacBook-Air-184 test_git_repo % git push origin main
The authenticity of host 'git.ligo.org (129.89.61.97)' can't be established.
ED25519 key fingerprint is SHA256:Rmg83x7s0+y61xu/kFU8eU1QL2o7e9bWJjrA5Tu3Kb4.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:49: gitlab.ligo.org
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'git.ligo.org' (ED25519) to the list of known hosts.
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 244 bytes | 244.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote:
remote: The private project lucy.thomas/test_git_repo was successfully created.
remote:
remote: To configure the remote, run:
remote:   git remote add origin git@git.ligo.org:lucy.thomas/test_git_repo.git
remote:
remote: To view the project, visit:
remote:   https://git.ligo.org/lucy.thomas/test_git_repo
remote:
remote:
remote: To git.ligo.org:lucy.thomas/test_git_repo.git
* [new branch]      main -> main
```

We can then push to the remote repository. It might ask us to add this to the list of known hosts, just type 'yes' and press enter



4. Creating A Project: Example



Lucy Thomas / test_git_repo

test_git_repo

main test_git_repo +

History Find file Edit Code

First commit, added test_file.txt
Lucy Thomas authored 4 hours ago

a3a65589

Name	Last commit	Last update
test_file.txt	First commit, added test_file.txt	4 hours ago

Project information

- 1 Commit
- 1 Branch
- 0 Tags
- 230 B Project Storage

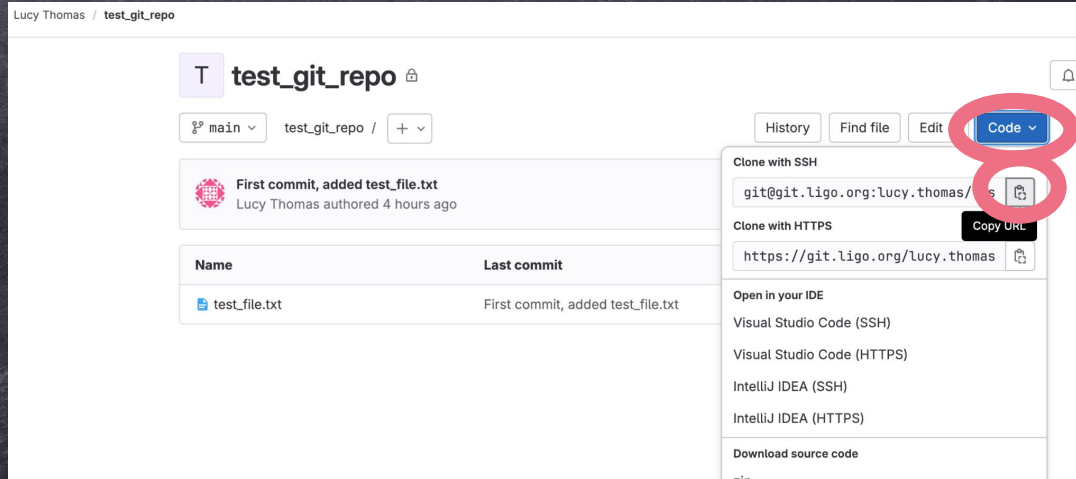
- + Add README
- + Add LICENSE
- + Add CHANGELOG
- + Add CONTRIBUTING
- + Enable Auto DevOps
- + Add Kubernetes cluster
- + Set up CI/CD
- + Add Wiki
- + Configure Integrations

Created on
June 24, 2024

We can now see our project as a remote repository in the LIGO GitLab instance!



4. Creating A Project: Example



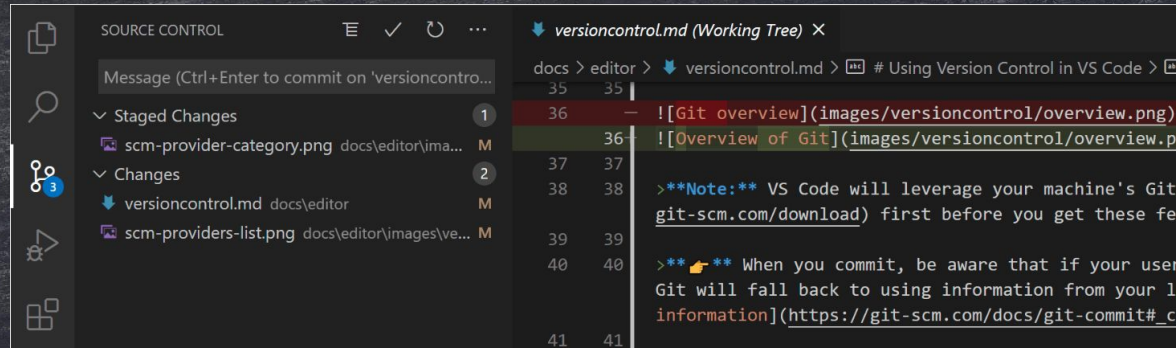
Now if others want to work on our repository (or if we want to work on it on another machine), we can clone the repository by copying and pasting this command, and then staging, committing and pushing as before

```
% git clone git@git.ligo.org:lucy.thomas/test_git_repo.git
```



4. (BONUS: Configuring VSCode To work with Git)

<https://code.visualstudio.com/docs/sourcecontrol/overview>



VSCode largely removes the need to type out the commands in your terminal/shell

A Few Final Points And Tips

- If you want to keep notes and documentation inside the repository, consider adding a **wiki** to the remote repository (instructions for GitLab: <https://docs.gitlab.com/ee/user/project/wiki/>)
- You can also make use of **issues** in a GitLab remote repository, to track tasks to do and help you project manage (GitLab documentation: <https://docs.gitlab.com/ee/user/project/issues/>)
- Don't store huge data files in the same repository as code files. **Git LFS** (Large File Storage) exists to manage large files (>~10Mb), which should be stored in separate LFS repositories (GitLab documentation: <https://docs.gitlab.com/ee/topics/git/lfs/>)
- If you can't remember a command or don't know the right one to use, ask someone or look it up! **ChatGPT**, **Stack Exchange** are very useful, as well as the **resources** on the next slide. Here's a package with some useful git tips which I sometimes use: <https://github.com/git-tips/tips>

Useful Resources

https://up1.github.io/git-guide/index.html	Great one-page website with useful explanations of basic commands
https://neurathsboat.blog/post/git-intro/#a-more-detailed-description-of-git-plumbing	Git for scientists, a useful blog post with good explanations of some basic commands
https://code.visualstudio.com/docs/sourcecontrol/overview	Documentation on how to use Git within the VSCode IDE (assumes you have some familiarity with Git)
https://training.github.com/downloads/git-hub-git-cheat-sheet.pdf	Git command cheatsheet, VERY USEFUL!
https://computing.docs.ligo.org/guide/gitlab/	IGVWN computing guide information about the LIGO GitLab instance

Thanks For
Listening, Happy
Coding!

THIS IS GIT. IT TRACKS COLLABORATIVE WORK
ON PROJECTS THROUGH A BEAUTIFUL
DISTRIBUTED GRAPH THEORY TREE MODEL.

COOL. HOW DO WE USE IT?

NO IDEA. JUST MEMORIZE THESE SHELL
COMMANDS AND TYPE THEM TO SYNC UP.
IF YOU GET ERRORS, SAVE YOUR WORK
ELSEWHERE, DELETE THE PROJECT,
AND DOWNLOAD A FRESH COPY.

